Noetia — Project Management Documentation

Version 1.0 | May 2026

---

1. Project Charter

Project Information

Field	Value
Project Name	Noetia — Multimodal Reading Platform
Start Date	June 2, 2025
End Date	May 25, 2026
Duration	12 months
Status	✅ Beta launched, App Store submission in progress

Project Objectives

1. Design, build, and launch a production-ready multimodal reading platform for Spanish-speaking audiences
2. Deliver phrase-by-phrase text-audio synchronization across web and mobile
3. Build a monetization engine (subscriptions + tokens) capable of processing real payments via Stripe
4. Launch a social layer (Clubs, fragment sharing, quote card generation) that creates user retention loops
5. Deploy to production infrastructure with CI/CD, monitoring, and automated backups

Success Criteria

Criterion	Target	Status
Production deployment live	noetia.app	✅ Achieved
Unit test coverage	≥ 80% across all services	✅ Achieved
Stripe payment processing	End-to-end, real transactions	✅ Achieved
Mobile apps buildable	iOS + Android via EAS	✅ Achieved
Database migrations applied	55 migrations, zero rollbacks	✅ Achieved
Monitoring and alerting	Grafana + Prometheus live	✅ Achieved
CI/CD pipeline	Auto-deploy on push to main	✅ Achieved

Constraints

- **Budget:** Bootstrapped (no external funding). Infrastructure cost < $50/month.
- **Team:** 4 people (PM, Backend, Frontend, DevOps) — no additional contractors.
- **Timeline:** Hard deadline of 12 months for beta-ready product.

- **Technology:** Must support offline use on mobile, real-time club features, and automated deployments.

---

2. Team Roles & Responsibilities

Product Manager (PM)

Core responsibilities:

- Define and maintain the product roadmap
- Write user stories and acceptance criteria for each sprint
- Run all Scrum ceremonies (planning, review, retrospective, daily standup)
- Manage the product backlog (priority ordering, estimation facilitation)
- Communicate progress to stakeholders
- Define and track KPIs
- Make product decisions on UX, feature scope, and technical trade-offs

Key decisions made:

- Chose Audible-style token model over pure subscription (enables per-title pricing without limiting free catalog)
- Prioritized Clubs feature in Q4 despite scope risk — correctly identified it as the primary retention driver
- Kept reading statistics as a Duolingo-inspired engagement loop (streak + weekly goals)
- Defined the 3-priority hierarchy: Reader experience > Author experience > Free library

---

Backend Developer

Core responsibilities:

- Architect and implement the NestJS API (REST endpoints, middleware, guards)
- Design PostgreSQL schema and write all TypeORM migrations
- Implement authentication (JWT, Google, Facebook, Apple OAuth)
- Integrate Stripe (plans, webhooks, checkouts, subscriptions, tokens, gift cards)
- Build the reading statistics heartbeat system and streak calculation
- Implement Clubs backend (7 tables: clubs, members, books, messages, discussions, polls, sessions)
- Write unit tests for all services (≥ 80% coverage)

Stack owned: NestJS, TypeORM, PostgreSQL, Redis, Stripe SDK, Nodemailer, Meilisearch

---

Frontend Developer

Core responsibilities:

- Build the Next.js web application (reader, library, fragments, sharing, profile, billing)
- Build the React Native mobile app (iOS + Android) with full feature parity
- Implement phrase-by-phrase synchronization UI (active phrase highlight, auto-scroll)
- Build the quote card share modal (4 platforms × multiple formats)
- Implement i18n system (Spanish/English) across web and mobile with API sync
- Integrate Expo EAS for mobile builds and OTA updates
- Write unit tests for all frontend components and hooks

Stack owned: Next.js 14, React Native (Expo SDK 51), Tailwind CSS, React Navigation, Zustand, AsyncStorage

---

DevOps Engineer

Core responsibilities:

- Provision and harden the production server (Contabo VPS, Ubuntu 24.04)
- Configure Traefik v2.11 as reverse proxy with automatic Let's Encrypt SSL
- Write and maintain all Docker Compose configurations (dev, prod overlay, server)
- Build the GitHub Actions CI/CD pipeline (test → build → deploy → migrate)
- Configure Grafana + Prometheus monitoring (API latency, error rate, container health)
- Set up automated database backups (PostgreSQL daily, MinIO weekly)
- Manage server security (UFW firewall, fail2ban, non-standard SSH port 222)
- Configure MinIO buckets (private books/audio, public images) and bucket policies

Stack owned: Docker, Docker Compose, Traefik, GitHub Actions, Grafana, Prometheus, PostgreSQL cron, MinIO

---

3. Project Methodology

Framework: Scrum with 2-week sprints **Sprint duration:** 14 days **Total sprints:** 24 sprints over 12 months **Sprint cadence:** Monday start, Friday end (with review/retro on the last Friday)

Ceremonies

Ceremony	Frequency	Duration	Participants
Daily Standup	Every weekday	15 min	All 4
Sprint Planning	Every 2 weeks (Monday)	2 hours	All 4
Sprint Review	Every 2 weeks (Friday)	1 hour	All 4
Sprint Retrospective	Every 2 weeks (Friday)	45 min	All 4
Backlog Refinement	Weekly (Wednesday)	1 hour	PM + relevant dev
Architecture Review	Monthly	2 hours	Backend + DevOps + PM

Tools

Tool	Purpose
GitHub Projects	Kanban board, issue tracking, sprint backlog
GitHub	Code repository, PR reviews, CI/CD
Slack	Daily communication, standup threads, incident alerts
Figma	UI design, wireframes, component specs
Notion	Project wiki, meeting notes, retrospective logs
Loom	Async video updates for PM → stakeholder comms

Definition of Done (DoD)

A story is **done** when:

- [ ] Feature is implemented and working as specified in the acceptance criteria
- [ ] Unit tests exist at the mirrored path under `tests/unit/`
- [ ] Tests cover happy path, edge cases, and error scenarios
- [ ] All tests pass (`npm test` / `pytest`)
- [ ] Coverage for the modified service is ≥ 80%
- [ ] Code has been reviewed and approved by at least one other team member
- [ ] No new lint errors or TypeScript errors introduced
- [ ] Feature is deployed to production (via CI/CD auto-deploy on merge to main)
- [ ] PM has verified the feature against acceptance criteria

---

4. Sprint History & Milestones

Q1: Foundation (Sprints 1–6 | Jun–Aug 2025)

Sprint 1–2 (Jun 2–27) *Goal: Project scaffolding and developer environment*

- Monorepo structure, Docker Compose (all 7 services), .env.example
- PostgreSQL init.sql, Redis config, MinIO bucket setup
- NestJS project init, TypeORM config, first migration (CreateUsersTable)
- Jest and pytest configuration across all services (80% coverage gates)
- GitHub Actions: lint + test + build CI on PRs

Sprint 3–4 (Jun 30–Jul 25) *Goal: Authentication system*

- Email + password auth (register, login, JWT access + refresh tokens)
- Email confirmation on registration (24h token via Resend SMTP)
- Password recovery via reset link
- Google OAuth, Facebook OAuth, Apple Sign-In
- Auth guards, logout, protected route wrapper (web)

Sprint 5–6 (Jul 28–Aug 22) *Goal: Books and reader foundation*

- Books table, CRUD endpoints, MinIO upload for text and audio
- Audio streaming endpoint (range requests)
- Phrase-level sync map design and storage
- Reading progress endpoint (save/restore by phrase index)
- Library page (book grid), book detail page
- Reader layout: text + audio controls, phrase rendering

Q1 Milestone: Working reader prototype ✅

Q2: Core Platform (Sprints 7–12 | Sep–Nov 2025)

Sprint 7–8 (Aug 25–Sep 19) *Goal: Full reading engine*

- Phrase highlight: audio time → active phrase synchronization
- Click phrase → seek audio
- Seamless mode switches (reading ↔ listening ↔ active listening)
- Active listening mode (audio + text + live highlight)
- Font size controls, dark mode, chapter navigation

- Reading preferences (localStorage persistence)

Sprint 9–10 (Sep 22–Oct 17) *Goal: Fragment & sharing system*

- Fragment CRUD, Fragment Sheet panel
- Text selection handler (mouse + touch)
- Quote card image generation service (Python Flask + Pillow)
- BullMQ worker for async image rendering
- LinkedIn, Instagram, Facebook, Pinterest templates

Sprint 11–12 (Oct 20–Nov 14) *Goal: Monetization foundation*

- Stripe integration: customer creation, Checkout sessions
- Subscription plans (Individual, Duo, Family — monthly + annual)
- Stripe webhooks: activate/cancel subscriptions, issue tokens on invoice.paid
- Token system: creditBalance, redemption, 90-day expiry
- Book purchases and access guard
- Pricing page, billing management UI

Q2 Milestone: Monetization live, first test payments processed ✅

Q3: Mobile & Social (Sprints 13–18 | Dec 2025–Feb 2026)

Sprint 13–14 (Nov 17–Dec 12) *Goal: React Native app shell*

- Expo SDK 51 project init, EAS project configuration
- Auth screens: login, register, social login (iOS + Android)
- Bottom tab navigation, library screen, book detail
- Reader screen: text + phrase sync on mobile
- google-services.json, Firebase setup for FCM

Sprint 15–16 (Dec 15–Jan 9) *Goal: Mobile features*

- Mobile fragment capture and Fragment Sheet
- Mobile paywall enforcement (subscription check on every login)
- Offline book download (phrases cached to AsyncStorage)
- Auto-sync on reconnect (NetInfo offline→online hook)
- Push notifications (Expo Notifications, APNs + FCM)
- Mobile audio player (Expo AV, background mode, speed control)

Sprint 17–18 (Jan 12–Feb 6) *Goal: Author portal + social sharing*

- Author/publisher registration and dashboard
- Book upload endpoint (text + audio + metadata + cover)
- Hosting tier enforcement (Basic: 1, Starter: 3, Pro: 12 books)
- Social OAuth account linking (LinkedIn, Facebook, Instagram, Pinterest)
- Direct publish from app to social platforms
- ShareModal: platform selector, format picker, hex picker, gradient directions

Q3 Milestone: iOS and Android builds running on physical devices ✅

Q4: Clubs, i18n, Stats & Launch (Sprints 19–24 | Mar–May 2026)

Sprint 19–20 (Feb 9–Mar 6) *Goal: Noetia Clubs — backend*

- 7 migrations: clubs, club_members, club_books, club_messages, club_discussions, club_polls+votes, club_sessions
- Club CRUD (create, join, leave, settings, ban)
- Phrase-anchored discussions tied to sync map phraseIndex
- Club polls for book nominations
- Escucha Juntos: scheduled live listening sessions
- Club invitation link system

Sprint 21–22 (Mar 9–Apr 3) *Goal: Clubs UI + i18n*

- Clubs discovery page, club page (4 tabs: Chat, Discussion, Polls, Sessions)
- Clubs Tutorial bottom sheet, WelcomeSplash entry point
- Full Spanish/English i18n across web and mobile
  - Language selection on first launch
  - All 8 email templates bilingual (EN/ES)
  - ReaderTopBar, BottomNav, all 5 tutorials fully i18n
  - LanguageProvider syncs language to/from API on mount

Sprint 23–24 (Apr 6–May 25) *Goal: Stats, privacy, QA, launch prep*

- Reading statistics system (heartbeat, 7-day bar chart, streak, goals)
- Privacy settings (4 toggles with optimistic updates)
- Profile page tabs (Profile / Stats / Privacy)
- Grafana monitoring fixed (Tailscale access, false-positive alert fixes)
- CD pipeline hardened (concurrency groups, dynamic container cleanup)
- EAS build configuration (production profiles, autoIncrement, OTA channels)
- 55 migrations applied to production, beta launch

Q4 Milestone: Beta launched at https://noetia.app ✅

---

5. Key Project Metrics

Velocity (story points per sprint, average)

Quarter	Avg Velocity	Notes
Q1 (Sprints 1–6)	28 SP	Ramp-up; infrastructure-heavy
Q2 (Sprints 7–12)	38 SP	Full cadence; complex features
Q3 (Sprints 13–18)	42 SP	Peak velocity; mobile parallelism
Q4 (Sprints 19–24)	35 SP	Clubs complexity; QA overhead

Deliverable Counts

Artifact	Count
Database migrations	55
API endpoints	~85
Unit test files	35+

React components (web)	40+
React Native screens	12
i18n translation keys	~400
Docker services	9

---

6. Risk Register

Risk	Status	Resolution
Phrase-sync complexity underestimated	Mitigated	Allocated extra Sprint 7–8 buffer
Stripe webhook reliability	Mitigated	Idempotency keys on all webhook handlers
Mobile offline-sync edge cases	Mitigated	NetInfo + retry queue implemented
Docker cache causing stale migrations	Hit	Added --no-cache build step; documented in runbook
SSH terminal corruption on server (multi-line paste)	Hit	Established nano/base64 protocol; added to CLAUDE.md
Grafana 404 after server restart	Hit	Cookie injection workaround; Tailscale access configured
Apple Sign-In differences iOS/Android	Mitigated	expo-apple-authentication handles natively

---

7. Retrospective Highlights

What went well

- **Mono-repo structure with Docker Compose** made cross-service changes fast and consistent
- **CI/CD from Sprint 1** meant every merge was deployable — no "integration week" at the end
- **80% test coverage gate** caught multiple regressions that would have shipped to production
- **Migration-first database changes** meant zero schema drift between environments
- **TypeScript everywhere** (API + web + mobile) reduced cross-layer bugs dramatically

What could be improved

- Clubs scope was larger than estimated — should have been split into two phases with a separate "Clubs v2" backlog
- i18n should have been designed into the UI from Sprint 1, not retrofitted in Q4
- Mobile development started in Q3; earlier mobile consideration would have influenced API design
- DevOps should have configured Grafana alerting tuning earlier to avoid false positives

Process improvements implemented

- Added architecture review meeting after the Clubs backend surfaced 7 interdependent migrations
- Introduced async Loom standup option to reduce meeting fatigue in Q4
- Created CLAUDE.md as a living developer guide — became the team's primary onboarding document

---

Document maintained by the Product Manager. Last updated: May 25, 2026.